

Requisitos - Final

Requisitos Funcionais (RF)

Ações e funcionalidades diretas que o sistema deve executar.

- **RF-01:** O Sistema deve autenticar o Usuário.
- **RF-02:** O Sistema deve perguntar ao Usuário se deseja uma solução para o erro descoberto.
- **RF-03:** O Sistema deve receber o caminho da pasta de Logs da Aplicação configurada.
- **RF-04:** O Sistema deve notificar os erros encontrados na varredura.
- **RF-05:** O Sistema deve alertar possíveis situações críticas descobertas pelo SDK.
- **RF-06:** O Sistema deve apresentar um histórico de erros.
- **RF-07:** O Sistema deve identificar os logs de erros, mesmo que não possuam padrões estruturados.
- **RF-08:** O Usuário deve configurar um Contexto detalhando a área de atuação de sua empresa.
- **RF-09:** O Sistema aplicará o contexto configurado no momento da varredura.
- **RF-10:** O Sistema deve fazer uma varredura periódica nos logs.
- **RF-11:** O Usuário poderá alterar o contexto configurado a qualquer momento.
- **RF-12:** O Sistema deve disponibilizar uma SDK capaz de capturar e enviar os logs da aplicação do cliente.
- **RF-13:** O Sistema deve compartilhar o link do Agente LLM, contendo o contexto e os logs de erro para interação.
- **RF-14:** O Sistema deve permitir filtrar logs por múltiplas opções (ex: data, criticidade, sistema).
- **RF-15:** O Sistema deve gerir os Usuários cadastrados.
- **RF-16:** O Sistema deve gerenciar os tokens de integração da SDK.
- **RF-17:** O Sistema deve possuir um Painel (Dashboard) interativo.
- **RF-18:** O Sistema deve conter um gerenciamento de assinatura/planos do usuário.
- **RF-19:** O Sistema deve sugerir soluções para problemas recorrentes.
- **RF-20:** O Sistema deve conter uma página de relatório unificada para multi-sistemas.
- **RF-21:** O Sistema deve ter uma barra de pesquisa global para os erros já armazenados.
- **RF-22:** O Sistema deve ter um contador/agrupador de logs repetidos.
- **RF-23:** O Sistema deve categorizar erros por Status (Ex: Não Solucionado, Solucionado).
- **RF-24:** O Sistema deve exportar relatórios para PDF e Excel.

- **RF-25:** O Sistema deve validar e mascarar se há senhas/credenciais vazadas nos logs.
- **RF-26:** O Sistema deve possuir um mecanismo de confirmação de leitura das notificações.
- **RF-27:** O Sistema deve permitir o arquivamento de erros.
- **RF-28:** O Sistema deve ser integrado a uma API de pagamento (ex: Mercado Pago, Abacate Pay).
- **RF-29:** O Sistema deve possuir uma interface mobile (aplicativo).
- **RF-30:** O aplicativo mobile deve notificar falhas via push.
- **RF-31:** O aplicativo deve conter um chat integrado para facilitar a comunicação com o LLM.
- **RF-32:** O aplicativo deve possuir um espelho simplificado do dashboard do sistema web.
- **RF-33:** O aplicativo deve ser lançado para Android e iOS.
- **RF-34:** O aplicativo deve suportar autenticação biométrica (Face ID / Touch ID).
- **RF-35:** O aplicativo deve suportar e apoiar a Autenticação de Dois Fatores (2FA).
- **RF-36:** O Sistema despacha um evento de **E-mail**, detalhando os erros críticos encontrados na varredura

Requisitos Não Funcionais (RNF)

Critérios de qualidade, arquitetura, desempenho e segurança.

- **RNF-01:** A conexão entre a SDK e o Core deve ser obrigatoriamente criptografada (Túnel de Criptografia).
- **RNF-02:** A primeira versão da SDK deve ser desenvolvida em Java, com previsão para expansão a outras linguagens.
- **RNF-03:** O Sistema utilizará a plataforma externa Clerk para gestão de usuários, garantindo maior segurança e escalabilidade.
- **RNF-04:** O banco de dados principal será o MongoDB, visando maleabilidade no armazenamento de logs não estruturados.
- **RNF-05:** O Sistema adotará a Arquitetura Hexagonal (Ports and Adapters).
- **RNF-06:** O Sistema deve salvar logs temporariamente (retenção de 7 dias dependendo do plano).
- **RNF-07:** A API de recebimento deve ser focada em alta performance e utilizar Webhooks para evitar travamento na aplicação cliente (SDK).
- **RNF-08:** O Sistema deve ser capaz de varrer e processar múltiplos formatos de arquivos de logs.
- **RNF-09:** O desenvolvimento deve ser estritamente orientado a Padrões de Projeto (Design Patterns).

- **RNF-10:** O frontend Web será desenvolvido em React (ou React Native Web), visando otimização e padronização da equipe. *(Nota: Vi que sugeriu React Native para web; é válido se usarem bibliotecas de unificação, embora React JS seja o padrão. É uma escolha interessante considerando que sua bagagem mobile anterior era forte em Flutter!)*
- **RNF-11:** O aplicativo Mobile será desenvolvido em React Native.
- **RNF-12:** O aplicativo e o sistema web devem utilizar paginação e dados otimizados para reduzir o PLT (Page Load Time).
- **RNF-13:** Os repositórios e a gestão das equipes de Mobile e Web devem ser separados.